
Compliance–Correctness Decomposition for Post-Training Evaluation of Small Language Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

Automatic evaluation of language models often conflates output compliance with task correctness, although these properties can change differently after post training. This makes strict accuracy difficult to interpret when fine tuning changes both response format and task performance. We identify this gap and introduce the *Compliance–Correctness Decomposition (CCD)*, an item paired framework that separates responses into invalid, valid but wrong, and valid and correct states. CCD requires no additional model generations and enables direct comparison of compliance, strict success, and the valid but wrong rate before and after supervised fine tuning. Across 49,140 matched response pairs spanning 14 small language models, four benchmarks, and nine inference configurations, compliance increases by 34.78 percentage points, while strict success increases by 12.12 points. The resulting 22.66 point increase in valid but wrong responses is consistent across the evaluated models, benchmarks, and inference configurations. On MBPP, compliance increases by 27.21 points, whereas execution based correctness decreases from 18.99% to 15.33%. These results indicate that improvements in evaluator facing compliance do not necessarily correspond to improved task correctness, supporting separate reporting of compliance and correctness in post training evaluation.

1 Introduction

Automatic benchmarks are a primary way to measure whether post-training improves language models, but their results depend on evaluation procedures and metrics [17]. For free-form generation, benchmark scores often depend on more than correctness. Responses must first satisfy an evaluator-facing format, such as a designated answer marker, choice label, or executable code. Strict accuracy therefore combines *compliance* with the expected format and *correctness* of the answer. These behaviors need not improve together. Prior work shows that prompt formatting and output representation can substantially affect measured performance [28, 20, 25], while instruction and format following can be evaluated separately from content quality [34, 31]. This distinction matters after supervised fine-tuning (SFT), which can establish new response conventions from limited supervision [33, 21]. Format-specific fine-tuning can likewise improve measured multiple-choice performance by teaching models to produce evaluator-accepted outputs [5].

Despite this evidence, post-training evaluations often report only the final benchmark score. An observed gain may therefore reflect more correct answers, better evaluator compliance, or both, which strict accuracy cannot distinguish. Grammar-constrained decoding can enforce valid structures [13, 2], but it changes generation and does not explain compliance and correctness changes in an existing pre/post evaluation. Recent work has also separated task solving from output formatting and studied correctness alongside structured-output usability [7, 11]. These studies motivate a complementary paired analysis of directly observed evaluator outcomes before and after post-training.

The closest studies address this measurement problem from complementary directions. Performance with and without format adherence has been compared across output representations [20]. Format-specific fine-tuning has been studied for multiple-choice evaluation [5], while task solving has been separated from output formatting during decoding [7]. Correctness and structured-output usability have also been examined in small language models [11], together with the validity–correctness trade-off introduced by hard output constraints [26]. These works vary output format, training objective, prompting, or decoding. Appendix Table 5 positions CCD relative to these closely related evaluation and structured-output studies.

Motivated by this need, we introduce the *Compliance–Correctness Decomposition* (CCD) to address a central ambiguity in strict benchmark scores: a change in success may reflect either better compliance with the evaluator or better task performance. CCD separates these outcomes using matched pre- and post-SFT responses evaluated under the same procedure. Stronger correctness instruments, such as execution-based evaluation for generated code [1, 19], are reported separately rather than folded into the strict outcome.

Our main contributions are as follows:

1. We introduce **Compliance–Correctness Decomposition (CCD)**, a simple item-paired framework that separates invalid, valid-but-wrong, and valid-and-correct responses, making changes in compliance and strict success directly measurable.
2. We audit **49,140 matched responses** across 14 models, four benchmarks, and nine inference configurations, finding that compliance improves nearly $3\times$ more than strict success after SFT.
3. We evaluate correctness beyond the strict evaluator and show on **MBPP** that higher compliance can coincide with lower execution-based functional correctness.
4. We propose **separate reporting of compliance and correctness** for post-training evaluation and release the paired records and evaluation code needed to reproduce our analysis.

We apply CCD to 49,140 matched response pairs across 14 four-bit-loaded instruction-tuned models, four benchmarks, and nine inference configurations. Across 495 evaluation cells, compliance increases by 34.78 percentage points, compared with a 12.12-point increase in strict success, yielding a 22.66-point increase in valid-but-wrong responses. On MBPP, compliance increases by 27.21 points, while execution-based functional correctness decreases from 18.99% to 15.33%. These results show that improved evaluator-facing compliance can occur without comparable gains in correctness and, in some settings, can coincide with lower functional correctness. We therefore argue for reporting compliance and correctness separately when evaluating post-training.

2 Methodology

We introduce the Compliance–Correctness Decomposition (CCD) as a post-hoc audit of evaluator outcomes before and after supervised fine-tuning (SFT). Rather than attributing score changes to a particular internal mechanism, we ask how the observed outcomes change under the same evaluation procedure. We separate each response into compliance with the evaluator’s output contract and strict success under the original correctness check, then compare these outcomes for the same items before and after SFT. We keep parser-derived and execution-based measures separate and use them only as secondary correctness instruments.

2.1 Supervised Fine-Tuning and Paired Generation

Post-training intervention. We adapt four-bit-loaded language models with low-rank adapters [8]. Training examples follow each model’s chat template and use the same “Final Answer:” convention as evaluation. We optimize the mean assistant-token negative log likelihood:

$$\mathcal{L}_{\text{SFT}}(\alpha) = -\sum_{z \in \mathcal{B}} \sum_{j \in A(z)} \log p_{\theta_0, \alpha}(z_j \mid z_{<j}) / \sum_{z \in \mathcal{B}} |A(z)|, \quad (2)$$

where θ_0 is frozen and α are trainable adapter parameters. Prompt and padding tokens are excluded. We use rank 16 adapters in attention and feed-forward projections; the recoverable run settings are reported in Appendix C.7.1. Since targets follow the evaluation convention, SFT can affect both answers and compliance, which CCD measures separately.

Matched evaluation. Each cell $c = (m, d, k)$ specifies a model, task, and inference configuration. For each retained item i , we compare pre-SFT $r_{i,0}$ and post-SFT $r_{i,1}$ responses under identical prompts, demonstrations, decoding rules, and generation budgets. Only items observed at both

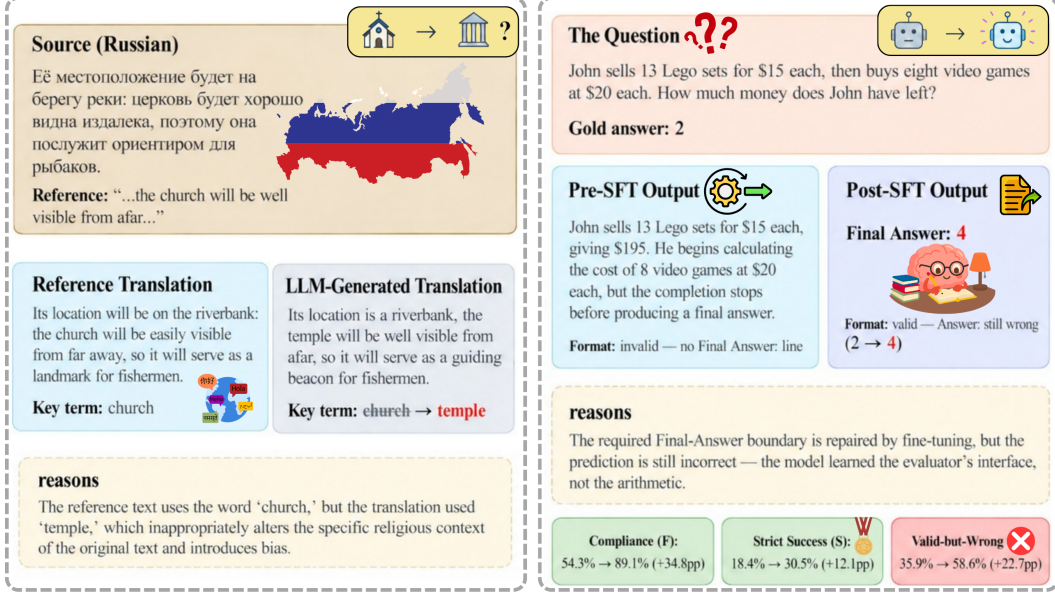


Figure 1: **Different evaluator blind spots can produce misleading scores.** (a) In machine translation, a meaning-changing substitution (*church* $\rightarrow$ *temple*) can remain hidden when evaluation emphasizes surface similarity. (b) In post-training evaluation, supervised fine-tuning repairs the required `Final Answer:` boundary for a GSM8K response, making it compliant even though its predicted answer is incorrect. CCD exposes this divergence by reporting compliance (F), strict success (S), and the valid-but-wrong rate ($F - S$) separately.

88 stages are retained. Direct, reasoning, and few-shot settings use greedy decoding; self-consistency
 89 uses sampled completions and the original harness aggregation. We retain its recorded compliance
 90 outcome without redefining the evaluator.

91 2.2 Separating Compliance from Strict Success

92 For each evaluation cell c , the evaluator first checks whether a response follows the required output
 93 contract, extracts an answer, and then evaluates that answer against the reference. Let ϕ_c , ϵ_c , and κ_c
 94 denote these three operations. For response $r_{i,t}$ and reference y_i , we define

$$F_{i,t} = \phi_c(r_{i,t}), \quad C_{i,t}^{\text{rel}} = \kappa_c(\epsilon_c(r_{i,t}), y_i), \quad S_{i,t} = F_{i,t} C_{i,t}^{\text{rel}}. \quad (3)$$

95 Here, F is **compliance** with the implemented output contract, C^{rel} is correctness under the archived
 96 parser and comparison rule, and S is **strict success**. Thus, strict success requires both a compliant
 97 response and a correct extracted answer.

98 Our primary analysis uses the recorded values of F and S without changing the evaluator. In
 99 particular, a response that fails the contract is not reclassified as correct simply because a fallback
 100 parser can recover an answer. Parser-derived correctness and execution-based correctness are instead
 101 treated as separate secondary measures. The output contracts are summarized in Table 6, and the
 102 exact extraction and normalization rules are given in Appendix D.

103 2.3 Decomposing Post-Training Change

104 Because $S_{i,t} \leq F_{i,t}$, every response belongs to one of three evaluator states:

$$Z_1 = (0, 0) \text{ invalid}, \quad Z_2 = (1, 0) \text{ valid but wrong}, \quad Z_3 = (1, 1) \text{ valid and correct}. \quad (4)$$

105 The labels *wrong* and *correct* refer only to the strict evaluator’s verdict.

106 For $X \in \{F, S\}$, let $\bar{X}_{c,t} = \frac{1}{n_c} \sum_{i \in \mathcal{I}_c} X_{i,t}$, $\Delta X_c = \bar{X}_{c,1} - \bar{X}_{c,0}$, $n_c = |\mathcal{I}_c|$. The rate of
 107 valid-but-wrong responses is $V_{c,t} = \bar{F}_{c,t} - \bar{S}_{c,t}$, since $F - S = 1$ exactly for state Z_2 . Therefore,

$$G_c \equiv \Delta F_c - \Delta S_c = \Delta V_c. \quad (5)$$

108 This identity holds for every evaluation cell. We refer to this quantity as the **compliance–success**
 109 **gap**. It measures the change in responses that satisfy the output contract but do not receive strict
 110 correctness credit.

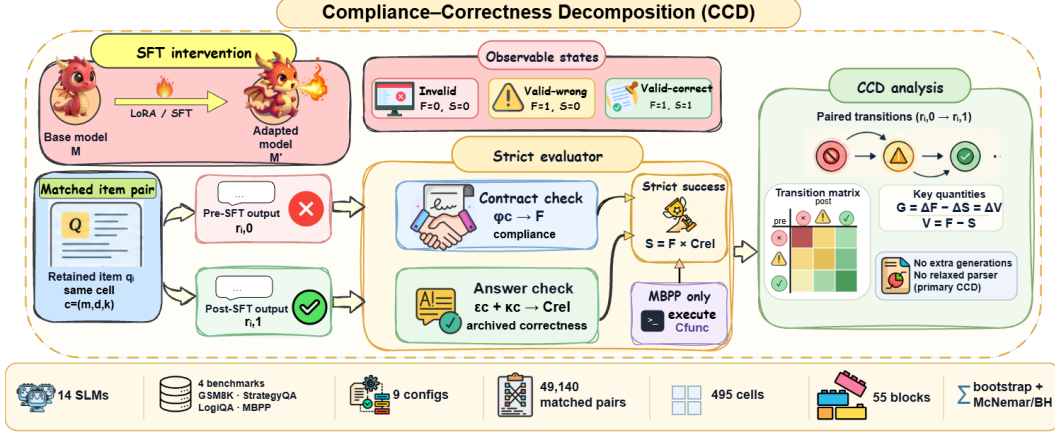


Figure 2: **Overview of the Compliance–Correctness Decomposition (CCD).** A base model and its SFT-adapted counterpart are evaluated on matched items within the same model–task–inference cell $c = (m, d, k)$. For each pre- and post-SFT response, the retained evaluator records compliance F and parser-derived correctness C^{rel} , with strict success defined as $S = FC^{\text{rel}}$. These outcomes produce three observable states: invalid, valid but wrong, and valid and correct. CCD accounts for paired transitions between these states and computes the exact compliance–success gap $G = \Delta F - \Delta S = \Delta(F - S)$. Execution-based correctness C^{func} is evaluated separately on MBPP. The audit covers 49,140 matched response pairs across 14 small language models, four benchmarks, nine inference configurations, 495 evaluation cells, and 55 model–task blocks.

111 The gap also has a simple transition interpretation. Let $N_{ab}^{(c)}$ denote the number of items moving
 112 from Z_a before SFT to Z_b after SFT. Then

$$n_c G_c = \underbrace{N_{12}^{(c)} - N_{21}^{(c)}}_{\text{net compliance gains without credit}} + \underbrace{N_{32}^{(c)} - N_{23}^{(c)}}_{\text{net credit losses among compliant outputs}}. \quad (6)$$

113 The first term captures responses that become compliant without becoming correct, while the second
 114 captures correct responses that lose strict credit while remaining compliant. Transitions from Z_1
 115 directly to Z_3 improve compliance and strict success equally and therefore do not contribute to the
 116 gap. Equations (5) and (6) are identities over the recorded evaluator outcomes. They describe where
 117 the observed score change occurs without attributing it to a particular internal mechanism or to latent
 118 reasoning ability.

119 2.4 Additional Correctness Measures and Paired Transitions

120 The three state decomposition does not assign correctness to invalid responses. To study correctness
 121 in more detail, we therefore use two additional measures. These measures provide useful information
 122 but require extra assumptions, so they remain secondary to the strict state analysis.

123 **Parser-derived correctness.** The archived measure C^{rel} applies the evaluator’s fallback extraction
 124 rules to noncompliant responses. This can show whether an answer can still be recovered when
 125 the required output format is not followed. However, its interpretation depends on the extractor.
 126 A number, option label, or code block recovered from a malformed response may not represent
 127 the model’s intended answer. In addition, parser-derived labels for noncompliant natural-language
 128 responses have not been validated against human judgments. The main results in Section 4.1 do not
 129 depend on these labels.

$$C_{i,t}^{\text{func}} = \mathbb{I} \left[\begin{array}{l} \text{the extracted program from } r_{i,t} \\ \text{passes every test in } T_i \end{array} \right]. \quad (7)$$

130 All selected outputs remain in the denominator. Empty or malformed programs, execution errors,
 131 timeouts, and failed tests receive zero. We compute C^{func} independently of compliance. Because the
 132 retained evaluators use different correctness procedures, including normalized source comparison and
 133 program execution, the archived C^{rel} labels are not uniformly functional judgments. We therefore

134 use C^{func} , rather than C^{rel} , for claims about program behavior. In general, FC^{func} need not equal
 135 the archived strict success indicator S .

136 **Four-state transition accounting.** For either secondary correctness measure $C \in \{C^{\text{rel}}, C^{\text{func}}\}$, the
 137 pair (C, F) defines four states: $E_1 = (0, 0)$, $E_2 = (0, 1)$, $E_3 = (1, 0)$, $E_4 = (1, 1)$.
 138 A 4×4 transition matrix records movement between these states for every matched item. Pure
 139 compliance gains change F from 0 to 1 while holding C fixed ($E_1 \rightarrow E_2$ or $E_3 \rightarrow E_4$); pure
 140 correctness gains change C from 0 to 1 while holding F fixed ($E_1 \rightarrow E_3$ or $E_2 \rightarrow E_4$). Joint changes,
 141 losses, and movements in opposing directions are retained rather than collapsed.

142 **Joint Success and Interaction.** Let $J_C = FC$ denote joint success under the chosen correctness
 143 measure. Suppressing item indices, we have

$$\Delta J_C = F_0 \Delta C + C_0 \Delta F + \Delta C \Delta F. \quad (8)$$

144 This identity makes the interaction between compliance and correctness explicit. Improving com-
 145 pliance alone increases joint success only when the item already receives correctness credit. When
 146 $C = C^{\text{rel}}$, $J_C = S$ by Equation (3), and merging E_1 with E_3 recovers the invalid strict state Z_1 .

147 **Sensitivity to fallback extraction.** We also bound the effect of assigning correctness credit to invalid
 148 responses. For any fallback extractor that agrees with the strict evaluator on compliant outputs, write
 149 $C^{\text{rel}} = S + R$, $R = \mathbb{I}[F = 0, C^{\text{rel}} = 1]$. Since $0 \leq R_{c,t} \leq 1 - \bar{F}_{c,t}$,

$$\Delta C_c^{\text{rel}} \leq \Delta S_c + 1 - \bar{F}_{c,1} \implies D_c^{\text{rel}} \geq G_c - (1 - \bar{F}_{c,1}), \quad (9)$$

150 where $D_c^{\text{rel}} = \Delta F_c - \Delta C_c^{\text{rel}}$. The same bound holds under common nonnegative aggregation weights.
 151 This gives a conservative bound on how much fallback credit can affect the parser-derived contrast
 152 without assuming that the recovered labels are correct.

153 2.5 Aggregation, Uncertainty, and Robustness

154 CCD computes changes within cells before aggregation. For $X \in \{F, S, V, C^{\text{rel}}, C^{\text{func}}\}$, we give
 155 each eligible cell equal weight:

$$\Delta X = \frac{1}{|\mathcal{C}_X|} \sum_{c \in \mathcal{C}_X} \Delta X_c, \quad G = \Delta F - \Delta S, \quad D^{\text{rel}} = \Delta F - \Delta C^{\text{rel}}. \quad (10)$$

156 Here, $\mathcal{C}_X$ contains cells where X is observed, with common cells and weights used for each contrast.
 157 Functional correctness uses the common execution-eligible subset. Equal-cell weighting prevents
 158 larger cells from dominating. We also report observation-weighted and equal-model estimates. Pooled
 159 transition counts directly describe item pairs and recover the observation-weighted gap.

160 **Uncertainty.** Inference configurations are repeated evaluations within model-task blocks, not inde-
 161 pendent units. We therefore use 10,000 hierarchical bootstrap replicates (seed 42), sampling models
 162 and their task blocks with replacement while retaining all configurations within each block. The 2.5th
 163 and 97.5th percentiles give 95% intervals. These quantify variation across the observed evaluation
 164 hierarchy, not training seeds; blocks sharing an adapter likewise do not show independent SFT runs.

165 **Paired significance tests.** We use exact McNemar tests for discordant paired outcomes within
 166 each cell [22], with Benjamini–Hochberg correction across cells for each outcome ($q < .05$) [3].
 167 Because cells may share models, tasks, or adapters, these tests are local diagnostics, not independent
 168 replications or aggregate inference.

169 **Robustness.** We vary aggregation weights, omit each model or task in turn, separate greedy and
 170 sampled decoding, and replace parser-derived with execution-based correctness where available. All
 171 statistics use retained paired records and require no additional generations.

172 3 Experiments

173 **Models and benchmarks.** We evaluate 14 instruction-tuned language models ranging from 270M
 174 to 8B parameters across four benchmarks: GSM8K, LogiQA, MBPP, and StrategyQA. These
 175 benchmarks cover numerical reasoning, multiple-choice reasoning, program synthesis, and Boolean
 176 reasoning, respectively, with different output contracts and correctness criteria. All models are
 177 evaluated before and after SFT using the same prompts, item sets, inference configurations, and
 178 evaluator versions within each comparison.

179 **Inference configurations.** For each model and benchmark, we consider nine inference configura-
 180 tions covering zero-shot, chain-of-thought, few-shot, and self-consistency settings. We retain only

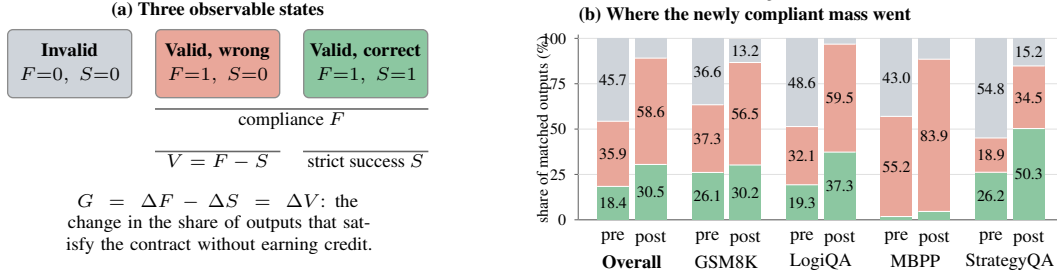


Figure 3: **The Compliance–Correctness Decomposition.** (a) Strict success S implies compliance F , so every output is invalid, valid but wrong, or valid and correct, and the valid-but-wrong rate $V = F - S$ is observable without any relaxed parsing. (b) Equal-cell shares of the three states before and after SFT, pooled over 495 cells and by benchmark. SFT converts most invalid outputs into compliant ones, but the majority of the newly compliant mass lands in the valid-but-wrong state: pooled V grows from 35.9% to 58.6%.

item-level matches observed at both stages, giving 49,140 paired responses across 495 evaluation cells and 55 model-task blocks. This paired design ensures that changes in compliance and strict success are measured on the same items rather than on potentially different samples.

Evaluation protocol. The primary analysis uses the evaluator’s original output contract and correctness procedure. We preserve historical evaluator differences rather than modifying them to create a common definition after the fact. We additionally evaluate MBPP programs against a standardized execution-based test suite to provide a common functional correctness measure across historical evaluator versions.

Implementation and provenance. Detailed model configurations, item selection, evaluator rules, SFT settings, provenance checks, and the standardized MBPP evaluation protocol are provided in Appendix C. All primary analyses are computed from the retained paired records and require no additional model generations.

4 Experimental Findings

4.1 Overall Findings

SFT improves compliance more than correctness. Table 1 reports the primary equal-cell estimates, with Figure 3(b) visualizing the changes. SFT increases strict success from 18.35% to 30.47% (+12.12 pp; 95% CI [8.08, 16.56]), while compliance rises from 54.28% to 89.06% (+34.78 pp; CI [23.29, 47.82]). The valid-but-wrong rate consequently increases by 22.66 pp ([13.42, 33.66]), closing 76.1% of compliance headroom but only 14.8% of strict-success headroom. The pattern holds across all benchmarks, with the gap ranging from +15.59 pp on StrategyQA to +28.72 pp on MBPP. On MBPP, compliance gains 31.48 pp while archived strict success changes little. Thus, greater compliance does not necessarily translate into greater task success. Cell-level paired tests provide similar evidence (Table 9 in Appendix E.1). After FDR correction, compliance changes significantly in 290 of 495 cells (274 increases, 16 decreases), compared with 183 for strict success (176 increases, 7 decreases). Parser-derived correctness changes significantly in only 67 cells (Appendix E.1). Overall, strict scores substantially understate changes in evaluator-facing behavior, while the decomposition itself requires no relaxed parser.

Most gains are compliance without correctness. Equation (6) identifies where the gap comes from, and Table 2 shows this directly without any extractor. Of 22,418 outputs that were invalid before SFT, 17,641 become compliant, but 11,843 (67.1%) remain wrong. After accounting for reverse moves, this flow contributes 11,424 items (23.25 pp) to the gap. In contrast, compliant wrong answers gain only 291 net correct transitions (−0.59 pp), while 189 correct outputs lose compliance. Together these flows give $(11,424 - 291)/49,140 = 22.66$ pp, exactly matching the observation-weighted gap. Thus, the large compliance gain is driven primarily by outputs becoming valid without becoming correct.

The pattern persists under relaxed extraction. The four-state analysis in Figure 4 further separates invalid responses by parser-derived correctness. Half of all pairs (50.61%) remain in the same state, while the largest transition is $E_1 \rightarrow E_2$ (9,924 pairs, 20.2%), where SFT fixes the output format without fixing the answer. Pure compliance gains account for 24.88% of all pairs, compared with 6.37% pure correctness gains and 7.11% joint gains. Parser-derived correctness increases by

Table 1: **Primary equal-cell levels before and after SFT (%) and the compliance–success gap $G = \Delta F - \Delta S = \Delta V$ (pp; overall 95% CI [13.42, 33.66]).** On MBPP the archived strict flag inherits the original harness’s limited execution coverage (Section C.5); standardized re-execution results are in Section 4.3. Green shading (lower $\rightarrow$ higher) highlights larger compliance–success gaps G . Color denotes gap size, not better performance.

Benchmark	Cells	Compliance F		Strict success S		Valid but wrong V		G
		Pre	Post	Pre	Post	Pre	Post	
GSM8K	126	63.39	86.78	26.10	30.24	37.30	56.53	+19.24
LogiQA	117	51.38	96.76	19.33	37.26	32.05	59.50	+27.45
MBPP	126	56.97	88.45	1.81	4.57	55.16	83.88	+28.72
StrategyQA	126	45.17	84.82	26.24	50.31	18.93	34.51	+15.59
Overall	495	54.28	89.06	18.35	30.47	35.93	58.59	+22.66

Table 2: **Pooled strict-state transition matrix over 49,140 matched outputs (count and share of all pairs).** States use only F and S (Eq. 4), so no relaxed extraction is involved. By Eq. (6), the gap is $(N_{12} - N_{21}) + (N_{32} - N_{23}) = (11,843 - 419) + (2,586 - 2,877) = 11,133$ items, i.e. 22.66 pp. Green shading (lower $\rightarrow$ higher) highlights larger counts in the valid-but-wrong destination column.

Pre-SFT strict state	Post-SFT strict state			Total
	Z_1 invalid	Z_2 valid, wrong	Z_3 valid, correct	
Z_1 invalid	4,777 (9.7%)	11,843 (24.1%)	5,798 (11.8%)	22,418
Z_2 valid, wrong	419 (0.9%)	14,431 (29.4%)	2,877 (5.9%)	17,727
Z_3 valid, correct	189 (0.4%)	2,586 (5.3%)	6,220 (12.7%)	8,995
Total	5,385	28,860	14,895	49,140

only 3.57 pp (CI [0.85, 6.65]), despite the 34.78 pp compliance gain, giving $D_{\text{rel}} = 31.22$ pp (CI [19.47, 44.17]). The relaxed analysis therefore reinforces the strict result, while the main conclusion remains independent of parser-derived labels for invalid outputs.

4.2 Robustness Across Analysis Choices

The gap survives analysis choices. Table 3 shows that the strict gap remains positive under alternative weighting, decoding regimes, and model or benchmark exclusions, ranging from 20.01 to 26.22 pp. Leave-one-model-out estimates range from 18.84 to 24.27 pp, and leave-one-benchmark-out estimates from 20.59 to 25.08 pp (Table 15). At finer granularity, the gap is positive for all 14 models, four benchmarks, and nine inference configurations (Tables 14 and 12 in Appendix E.3). The same pattern holds for the parser-derived contrast.

The finding does not depend on the extractor. The primary strict analysis uses no relaxed extraction. More generally, Eq. (9) bounds the parser-derived gap at $D_{\text{rel}} \geq 11.72$ pp even under the most favorable possible treatment of invalid outputs. The documented extractor gives $D_{\text{rel}} = 31.22$ pp, while crediting every invalid output gives 57.45 pp; the strict boundary gives 22.66 pp (Appendix E.5). Thus, the qualitative conclusion holds across substantially different treatments of invalid responses.

The effect follows compliance headroom. Cells with very low initial compliance show the largest changes: the lowest quartile starts at 0.56% compliance, gains 71.61 pp, and has $D_{\text{rel}} = 66.63$ pp. At the highest quartile, starting at 99.52%, compliance changes by only -0.85 pp and $D_{\text{rel}} = -2.26$ pp. This shows that the decomposition is not a scale-free effect size, but tracks the available compliance headroom (Table 13). More broadly, compliance and parser-derived correctness are only weakly coupled across cells (Pearson $r = .28$), with compliance improving without a correctness gain in 149 of 495 cells (30.1%).

4.3 MBPP: Compliance Rises While Functional Correctness Falls

MBPP provides an independent correctness check because generated programs can be executed. Across all nine configurations, selected-output functional accuracy falls from 18.99% to 15.33% (-3.66 pp; model-cluster CI $[-7.71, -0.50]$), while compliance rises by 27.21 pp. Across the six deterministic configurations, pass@1 falls from 19.62 (-4.82 pp; CI $[-9.65, -1.05]$) against a 26.31 pp compliance gain. The compliance–function gap is positive in every configuration, and 64 of 117 cells (54.7%) improve in compliance without improving functional correctness (Appendix F). Configuration-level and checkpoint-level results are reported in Tables 23 and 24, respectively.

Pre-SFT state	Post-SFT state			
	E_1 wrong, invalid	E_2 wrong, valid	E_3 correct, invalid	E_4 correct, valid
E_1	3,228 6.6%	9,924 20.2%	251 0.5%	3,494 7.1%
E_2	368 0.7%	14,431 29.4%	51 0.1%	2,877 5.9%
E_3	308 0.6%	1,919 3.9%	990 2.0%	2,304 4.7%
E_4	111 0.2%	2,586 5.3%	78 0.2%	6,220 12.7%

■ pure compliance gain
 ■ pure correctness gain
 ■ joint gain
 ■ loss or trade-off
 ■ unchanged

Figure 4: **Pooled transition matrix over 49,140 matched outputs under the relaxed extractor.** Cells show count and share; rows are pre-SFT and columns post-SFT, with states (correct, valid). The dominant transition, $E_1 \rightarrow E_2$, becomes valid while remaining wrong. Non-MBPP invalid labels are parser-derived.

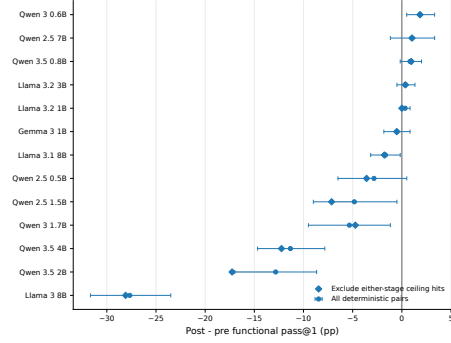


Figure 5: **Per-checkpoint change in deterministic MBPP pass@1 after SFT (pp).** Circles show all 600 pairs with paired-bootstrap 95% intervals; diamonds exclude generation-ceiling pairs. All checkpoints starting above 15% decline, while none starting below 5% does.

Table 3: **Robustness of the compliance–success gap (pp).** Strict columns use the evaluator’s own verdict; parser-derived columns use the relaxed extractor. The three largest MBPP decliners are Llama 3 8B, Qwen 3.5 2B, and Qwen 3.5 4B (Section 4.3); ranges are minima and maxima over the omitted units. Green shading (lower $\rightarrow$ higher) highlights larger values of the primary gap G ; other columns are unshaded.

Analysis	Cells	Strict evaluator (primary)			Parser-derived	
		ΔS	ΔF	G	ΔC_{rel}	D_{rel}
Primary, equal-cell weighting	495	+12.12	+34.78	+22.66	+3.57	+31.22
95% hierarchical-bootstrap CI		[8.08, 16.56]	[23.29, 47.82]	[13.42, 33.66]	[0.85, 6.65]	[19.47, 44.17]
Observation-weighted	495	+12.01	+34.66	+22.66	+3.56	+31.10
Equal-model weighted	495	+12.24	+35.37	+23.13	+3.56	+31.81
Excluding MBPP	369	+15.32	+35.91	+20.59	+4.91	+31.00
Excluding three largest MBPP decliners	387	+12.65	+38.87	+26.22	+2.61	+36.26
Excluding both	288	+15.93	+40.26	+24.33	+3.49	+36.78
Deterministic configurations only	330	+12.28	+36.27	+23.99	+3.42	+32.85
Self-consistency configurations only	165	+11.80	+31.81	+20.01	+3.85	+27.95
Leave one model out (range)	459–468	11.55–12.57	30.39–36.84	18.84–24.27	2.86–3.96	27.11–33.33
Leave one benchmark out (range)	369–378	8.04–15.32	31.51–38.68	20.59–25.08	1.63–4.91	29.56–34.47

Where functional correctness is lost. Of 2,222 pre-SFT passes, 1,014 become failures, while 586 failures become passes, giving a net loss of 428 passes (Table 4B). Assertion failures account for 260 of this loss (60.7%) and syntax errors for 140 (32.7%). Undefined-name errors instead decline by 47, suggesting that post-SFT programs become more self-contained without becoming more correct. Marginal execution outcomes are reported in Table 26. The qualitative examples in Appendix I show the same pattern: an initially passing program is replaced by one that better follows the output contract but fails execution.

The decline is not explained by evaluation artifacts. Every non-executable program remains a failure in the denominator, so malformed outputs are not discarded. Truncation also cannot explain the result: generation ceiling hits fall from 16.68% to 1.14%, and excluding every pair that hits the ceiling at either stage leaves 6,436 pairs with a larger decline of -5.47 pp (CI $[-10.57, -1.30]$).

The effect is heterogeneous. Figure 5 visualizes this heterogeneity; the plotted values are listed in Table 25. The decline is concentrated in checkpoints with meaningful pre-SFT functional accuracy. Llama 3 8B drops by 27.67 pp, Qwen 3.5 2B by 12.83 pp, and Qwen 3.5 4B by 11.33 pp, while seven checkpoints starting below 5% essentially flat. Only Qwen 3 0.6B shows a significant improvement (+1.83 pp; CI $[0.50, 3.33]$). Thus, across the completed interventions, learning the output contract coexists with lower functional correctness where there was meaningful correctness to lose. These records do not establish causality because we lack retraining seeds and supervision ablations.

A. Functional summary (% , changes in pp)					
Scope	Pairs	Pre	Post	ΔC_{func} [95% CI]	ΔF
All nine configurations (selected-output accuracy)	11,700	18.99	15.33	−3.66 [−7.71, −0.50]	+27.21
Six deterministic configurations (pass@1)	7,800	19.62	14.79	−4.82 [−9.65, −1.05]	+26.31
Deterministic, either-stage ceiling hits removed	6,436	21.73	16.26	−5.47 [−10.57, −1.30]	−
B. Paired pass/failure accounting (all 11,700 pairs)					
Failure class	Pass→fail		Fail→pass		Net loss
Assertion failure	525		265		+260
Syntax error	189		49		+140
TypeError	97		73		+24
IndexError	24		6		+18
AttributeError	16		0		+16
NameError	105		152		−47
All classes	1,014		586		+428

Table 4: **Execution-based MBPP results for the 13 checkpoints with standardized test mappings.** Panel A intervals are model-cluster bootstrap CIs; the third row removes every deterministic pair whose generation reached the model’s token ceiling at either stage. Panel B lists the failure classes with the largest paired imbalances; net counts are accounting quantities, not causal attributions. Only functional change in A and net loss in B are shaded. Green (lower → higher) encodes larger signed values within each panel, not statistical significance.

5 Discussion

Strict gains can overstate what post-training changes. We find that strict success combines evaluator compliance with task correctness, which can change at different rates. In our audit, SFT increased compliance by 34.78 pp but strict success by only 12.12 pp, leaving a 22.66 pp increase in valid-but-wrong responses. We therefore argue that higher benchmark scores can reflect substantial learning of the evaluator-facing contract without a comparable gain in task success. This does not make compliance learning undesirable; rather, we should report what drives the score change.

Correctness depends on the instrument. We find that strict evaluation provides the cleanest primary outcome, while relaxed extraction reveals additional transition structure but depends on the extractor. Where a behavioral oracle exists, we find that it provides a stronger capability check: on MBPP, compliance increased while functional correctness declined. We therefore view these measures as complementary rather than interchangeable estimates of correctness.

Implications for evaluation. We recommend that benchmarks with output contracts report pre- and post-training compliance F , strict success S , and valid-but-wrong rate $F - S$. We also recommend retaining item-level transitions and versioning contract checkers and relaxed extractors separately. Where a behavioral oracle exists, we suggest using it to anchor capability claims. These additions complement, rather than replace, strict accuracy.

Limitations. We study heterogeneous, single-run SFT interventions rather than a controlled experimental design. Complete training-dose metadata are available for only five of 55 model–task blocks, so our results do not isolate the effect of a specific SFT recipe or imply a universal format–capability trade-off. Since the output convention appears in the SFT targets, the observed compliance gains are also tied to the exposed contract. Controlled dose sweeps, training seeds, and supervision ablations would further separate interface learning from capability changes, forgetting, and overfitting. Also, parser-derived correctness for invalid non-MBPP outputs is not human-validated. MBPP uses the original benchmark assertions rather than augmented suites, and its generation-ceiling analysis uses a token-count proxy. Finally, our study covers four English benchmarks and models up to 8B parameters, so results may vary across languages, model scales, contracts, and post-training methods.

6 Conclusion

Strict accuracy remains an important metric, but it does not reveal why a score changes. CCD makes this distinction explicit by separating compliance, strict success, and the valid-but-wrong state using matched pre- and post-training outputs. Across 49,140 matched responses, compliance increased nearly three times as much as strict success, with a 22.66 pp increase in valid-but-wrong responses. On MBPP, an independent execution-based measure shows that higher compliance can coexist with lower functional correctness. Together, these results support a simple evaluation principle: *report whether models learned to satisfy the evaluator, became more correct, or both.*

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- [2] Debangshu Banerjee, Tarun Suresh, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. CRANE: Reasoning with constrained LLM generation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 2836–2857. PMLR, 2025. URL <https://proceedings.mlr.press/v267/banerjee25a.html>.
- [3] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1): 289–300, 1995.
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- [5] Alec Bunn, Sarah Wiegrefe, and Ben Bogin. Fine-tune on the format: First improving multiple-choice evaluation for intermediate LLM checkpoints. In *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM²)*, pages 511–521, Vienna, Austria and virtual meeting, July 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.gem-1.46/>.
- [6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [7] Haikang Deng, Po-Nien Kung, and Nanyun Peng. Decoupling task-solving and output formatting in LLM generation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16764–16781, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.764. URL <https://aclanthology.org/2026.acl-long.764/>.
- [8] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient fine-tuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, pages 10088–10115, 2023. doi: 10.52202/075280-0441. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html.
- [9] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. doi: 10.48550/arXiv.2407.21783. URL <https://arxiv.org/abs/2407.21783>.
- [10] Tingchen Fu, Deng Cai, Lemao Liu, Shuming Shi, and Rui Yan. Disperse-then-merge: Pushing the limits of instruction tuning via alignment tax reduction. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2967–2985, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.175. URL <https://aclanthology.org/2024.findings-acl.175/>.
- [11] Cosimo Galeone, Minsu Park, Giuseppe Ettorre, and Daniele Ligorio. When correct isn’t usable: Improving structured output reliability in small language models, 2026. URL <https://arxiv.org/abs/2605.02363>.

- [12] Gemma Team. Gemma 3 Technical Report. *arXiv preprint arXiv:2503.19786*, 2025. doi: 10.48550/arXiv.2503.19786. URL <https://arxiv.org/abs/2503.19786>.
- [13] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.674. URL <https://aclanthology.org/2023.emnlp-main.674/>.
- [14] Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9:346–361, 2021. doi: 10.1162/tacl_a_00370. URL <https://aclanthology.org/2021.tacl-1.21/>.
- [15] Yuto Harada, Yusuke Yamauchi, Yusuke Oda, Yohei Oseki, Yusuke Miyao, and Yu Takagi. Massive supervised fine-tuning experiments reveal how data, layer, and training factors shape LLM alignment quality. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 22360–22381, Suzhou, China, November 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.1138. URL <https://aclanthology.org/2025.emnlp-main.1138/>.
- [16] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, volume 35, pages 22199–22213, 2022. doi: 10.52202/068431-1613. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html.
- [17] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue Wang, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=iO4LZibEqW>.
- [18] Jian Liu, Leyang Cui, Hanmeng Liu, Dandan Huang, Yile Wang, and Yue Zhang. LogiQA: A challenge dataset for machine reading comprehension with logical reasoning. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, pages 3622–3628, 2020. doi: 10.24963/ijcai.2020/501. URL <https://www.ijcai.org/Proceedings/2020/501>.
- [19] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-0943. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract.html.
- [20] Do Xuan Long, Hai Nguyen Ngoc, Tivatis Sim, Hieu Dao, Shafiq Joty, Kenji Kawaguchi, Nancy F. Chen, and Min-Yen Kan. LLMs are biased towards output formats! systematically evaluating and mitigating output format bias of LLMs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 299–330, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.naacl-long.15. URL <https://aclanthology.org/2025.naacl-long.15/>.
- [21] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, and Adam Roberts. The flan collection: Designing data and methods for effective instruction tuning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 22631–22648. PMLR, 2023. URL <https://proceedings.mlr.press/v202/longpre23a.html>.

- [22] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- [23] Qwen Team. Qwen2.5 Technical Report, 2024. URL <https://arxiv.org/abs/2412.15115>.
- [24] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- [25] Rishi Ravikumar, Nuhu Ibrahim, and Riza Batista-Navarro. Lost in formatting: How output formats skew LLM performance on information extraction. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5498–5513, Rabat, Morocco, March 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.eacl-long.256. URL <https://aclanthology.org/2026.eacl-long.256/>.
- [26] Jaideep Ray. The constraint tax: Measuring validity–correctness tradeoffs in structured outputs for small language models. *arXiv preprint arXiv:2605.26128*, 2026. doi: 10.48550/arXiv.2605.26128. URL <https://arxiv.org/abs/2605.26128>. Preprint.
- [27] Victor Sanh, Albert Webson, Colin Raffel, Stephen H. Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, Manan Dey, M. Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal Nayak, Debajyoti Datta, Jonathan Chang, Mike Tian-Jian Jiang, Han Wang, Matteo Manica, Sheng Shen, Zheng Xin Yong, Harshit Pandey, Rachel Bawden, Thomas Wang, Trishala Neeraj, Jos Rozen, Abheesht Sharma, Andrea Santilli, Thibault Fevry, Jason Alan Fries, Ryan Teehan, Tali Bers, Stella Biderman, Leo Gao, Thomas Wolf, and Alexander M. Rush. Multitask prompted training enables zero-shot task generalization. In *The Tenth International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=9Vrb9D0W14>.
- [28] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=RLu5lyNXjT>.
- [29] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- [30] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *The Tenth International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=gEZrGCozdqR>.
- [31] Congying Xia, Chen Xing, Jiangshu Du, Xinyi Yang, Yihao Feng, Ran Xu, Wenpeng Yin, and Caiming Xiong. FOFO: A benchmark to evaluate LLMs’ format-following capability. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 680–699, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.40. URL <https://aclanthology.org/2024.acl-long.40/>.
- [32] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen3 Technical Report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [33] Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. LIMA: Less is more for alignment. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-2400. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/ac662d74829e4407ce1d126477f4a03a-Abstract-Conference.html.
- [34] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL <https://arxiv.org/abs/2311.07911>.

Table 5: **Positioning CCD relative to closely related evaluation and structured-output studies.** A check mark indicates that the feature is an explicit part of the study; a dash indicates that it is not a primary analysis. “Transitions” denotes matched, item-level changes between conditions, and “behavioral oracle” denotes execution-based correctness.

Work	Primary comparison	Contract fixed	Item-level transitions	Wrong-valid outcome	Behavioral oracle
FoFo [31]	Models across domain-specific formats	—	—	—	—
Long et al. [20]	Requested formats and mitigation methods	—	—	—	—
Bunn et al. [5]	Standard versus format-specific fine-tuning	✓	—	—	—
Deng et al. [7]	Standard versus decoupled decoding	✓	—	—	—
Galeone et al. [11]	Prompting and constrained-generation strategies	✓	—	—	—
Ray [26]	Prompt-only versus hard-schema control	✓	—	✓	✓
CCD (ours)	Matched pre- versus post-SFT responses	✓	✓	✓	✓

A Related Work

Format-aware and multidimensional evaluation. Recent work has shown that a single benchmark score can hide important differences in model behavior. HELM evaluates models across multiple scenarios and desiderata [17], while IFEval isolates objectively verifiable instruction following [34]. FoFo further shows that format following can vary independently of content quality [31]. Measured performance is also sensitive to how tasks and outputs are represented: meaning-preserving prompt changes can produce substantial score differences [28], and equivalent output formats can affect fine-tuned models [25]. Most closely related, prior work distinguishes performance with and without format adherence and studies the effect of requested output formats [20]. These works establish that evaluator-facing format is an important component of measurement. We focus on a narrower question: *when the evaluator and output contract are fixed, what accounts for a score change after post-training?*

Structured generation and post-training. A substantial body of work improves or enforces structured output through constrained decoding, prompting, or additional training. Grammar-based decoding can guarantee formal validity [13], while CRANE combines structural constraints with intermediate reasoning [2]. Other approaches fine-tune models for particular output formats [5], separate task solving from format control during decoding [7], or optimize prompts for structured output reliability [11]. Instruction tuning itself can improve instruction following and transfer [30, 27, 21], with LIMA showing that relatively small amounts of supervision can substantially shape response conventions [33]. Studies of SFT also report that its effects vary with training data, model, and training stage, and that gains on some objectives can coincide with losses on others [10, 15]. This literature motivates separating observable dimensions of post-training change, but does not provide an item-paired decomposition of compliance and strict correctness under a fixed evaluator.

Correctness beyond parsing. For some tasks, correctness can be assessed independently of whether the response follows the expected textual format. In code generation, MBPP uses test execution to evaluate generated programs [1], while EvalPlus shows that execution-based conclusions depend on the coverage and strength of the test suite [19]. Thus, parser-based validity and functional correctness are distinct measurement instruments. Our work retains this distinction: CCD uses the original evaluator to measure compliance and strict success, while execution-based correctness is treated as an independent secondary measure where available. In contrast to approaches that modify generation, relax the evaluator, or compare alternative formats, CCD asks how the same evaluator outcomes change across matched responses before and after SFT. This provides a direct account of whether post-training primarily changes compliance, strict success, or the population of responses that are compliant but still incorrect.

B Reproducibility statement

The accompanying reproducibility materials include retained pre- and post-SFT generations, 49,140 paired item-level records, cell-level results, execution labels for 11,700 paired MBPP programs, and documented evaluation settings. These materials support reproduction of the reported tables, figures, confidence intervals, transition matrices, and sensitivity analyses without repeating model training or generation. They also support verification of the strict-score identity (Eq. 3) and the paired flow

decomposition (Eq. 6). Exact reproduction of every historical SFT run remains limited by incomplete training metadata for 50 of the 55 model–task evaluation blocks (Appendix C.7.1).

The materials additionally include a stratified 300-item parser-audit sample and a two-annotator protocol. The sample contains 75 pre-SFT items per benchmark, balanced across extractor states where coverage permits. Human annotation and adjudication have not been completed; no agreement statistics or human-validated correctness claims are based on this sample.

Minimal replication. The headline result requires only that records be paired by stable item ID within each model $\times$ dataset $\times$ configuration cell, that F and S be computed at both stages, and that cell-level deltas be averaged under the stated weighting. The four-state analysis additionally requires the versioned relaxed-extractor output C_{rel} ; the MBPP analysis requires the execution status of each candidate. A replication should retain, for every item, the model label and loader ID, dataset and item ID, configuration, stage, raw prompt and completion, extracted answer, gold answer, format flag, correctness flag, strict flag, generated-token count, and, where applicable, execution status and error class; for self-consistency, the sampled candidates, their parsed answers and format flags, the selected winner, and the tie-breaking rule. Run-level records should include seed, maximum sequence length, maximum new tokens, training-set size, maximum steps, effective batch, learning rate, adapter configuration, and package versions.

C More Details on Evaluation Design

Here, we define how CCD decomposes post-training change. We now specify the evaluation design that makes this decomposition comparable across stages. Before- and after-SFT records must refer to the same item under the same prompt construction, inference procedure, and evaluator, while historical differences in evaluator versions and SFT recipes must remain visible. This design supports an audit of recorded outcomes across models, tasks, and inference settings without obscuring the scope of each claim.

C.1 Models, Benchmarks, and Inference Configurations

Models. We evaluate 14 instruction-tuned models spanning approximately 270M to 8B parameters. The model set comprises Gemma 3 at 270M and 1B parameters; Llama 3 at 8B, Llama 3.1 at 8B, and Llama 3.2 at 1B and 3B; Qwen 2.5 at 0.5B, 1.5B, and 7B; Qwen 3 at 0.6B and 1.7B; and Qwen 3.5 at 0.8B, 2B, and 4B [12, 9, 23, 24, 32]. Every run uses four-bit weight loading with low-rank adapters [8]. Table 7 lists the canonical loader identifier associated with each analytical model label; dataset-specific names retained in historical files are treated as provenance fields, not as additional models.

Benchmarks and output contracts. We use four benchmarks with different answer types and correctness mechanisms: GSM8K for numerical reasoning [6], LogiQA for multiple-choice logical reasoning [18], MBPP for Python program synthesis [1], and StrategyQA for implicit Boolean reasoning [14]. This variation is important for the paper’s central question. A compliance gain tied only to one answer representation would be qualitatively different from a pattern that recurs across numbers, option labels, Boolean answers, and programs.

Inference configurations. Each available model–task block is evaluated before and after SFT under nine prespecified configurations: direct zero-shot prompting (Zero), zero-shot chain-of-thought prompting (CoT; 16), few-shot prompting with $k \in \{0, 1, 3, 5\}$ demonstrations (FS $_k$; 4), and self-consistency with $k \in \{1, 3, 5\}$ sampled reasoning paths (SC $_k$; 29). FS $_0$ retains the few-shot scaffold with an empty demonstration block and is therefore distinct from Zero. Zero, CoT, and few-shot configurations use greedy decoding; self-consistency samples candidates and votes over their extracted answers. Pre-SFT and post-SFT refer to evaluation immediately before and after fine-tuning.

C.2 Paired Design and Item Selection

The archive yields 49,140 matched pre-/post-SFT response pairs distributed across 495 evaluation cells and 55 model–task blocks. One otherwise expected block, Qwen 2.5 7B on LogiQA, has no retained item-level records. Of the 495 observed cells, 477 contain 100 matched pairs and 18 contain 80; the smaller cells correspond to Qwen 2.5 7B on GSM8K and StrategyQA.

Table 6: **Output contracts and evaluation criteria.** Strict success S requires both compliance with the specified output contract and acceptance under the corresponding strict criterion (Equation (3)). Fallback extraction is used only for the secondary parser-derived analysis described in Section 2.4. MBPP functional claims use standardized execution C^{func} , rather than the archived correctness flag.

Benchmark	Output	Required contract	Strict criterion
GSM8K	Number	Final Answer: <number>	Normalized numeric match
StrategyQA	Yes/No	Final Answer: Yes No	Label match
LogiQA	Option	Final Answer: A B C D	Option match
MBPP	Program	Final-answer boundary followed by Python code	Archived evaluator verdict; standardized execution reported separately

Pairs are formed using stable task identifiers or explicit item indices whenever available. When neither is retained, we require agreement in item order, question text, and reference answer before accepting the alignment. Official training and evaluation partitions remain separate throughout. GSM8K and StrategyQA use the first n retained evaluation items, whereas LogiQA and MBPP are shuffled with seed 42 before item selection.

The historical runs generated up to 100 pre-SFT and 200 post-SFT records per configuration. We do not compare these unequal sets directly. Instead, each analysis cell uses only the identity-matched intersection of items present at both stages, preventing additional post-SFT records from changing the comparison set. This restriction also limits the estimand: request failures were not logged consistently enough to justify a missing-at-random assumption, so all reported changes are conditional on the retained pairs rather than estimates for unobserved generations.

C.3 Contract Checking and Parser-Derived Extraction

The strict evaluator operationalizes the two primary CCD variables. It sets compliance F from the required answer boundary and representation, and it assigns strict success S only when a compliant response also satisfies the task-specific criterion in Table 6. A contract-invalid response never receives strict credit, even if a secondary extractor can recover a plausible answer.

The parser-derived instrument C^{rel} applies a deterministic fallback when the required answer region is absent or unusable. For GSM8K, it takes the last normalized numeric match in the completion; for StrategyQA, it searches for a standalone Yes or No label; for LogiQA, it searches for an explicit option from A to D; and for MBPP, it uses the final fenced Python block or, if no such block exists, the completion itself. Appendix D specifies the exact normalization and matching rules. These fallback outputs support the secondary four-state analysis, but they do not alter F or the strict-success outcome.

Self-consistency requires additional version tracking. Each candidate is parsed separately before voting. Earlier harnesses mark the aggregate as compliant when any sample is compliant, even if it does not support the winner; later harnesses require a compliant sample among the winner’s supporters. Program voting likewise uses either normalized source text or abstract-syntax-tree normalization. We preserve each record’s original rule and use greedy-only analyses to test sensitivity to these differences.

All extractors are deterministic and versioned, and all 49,140 pairs satisfy the strict-success identity $S = FC^{\text{rel}}$ at both stages. Parser-derived labels for contract-invalid natural-language responses have not been validated against human judgments. The release includes a stratified 300-item sample—75 pre-SFT items per benchmark, balanced across extractor states where possible—and a two-annotator protocol, but annotation and adjudication are incomplete. We therefore make no human-validated correctness or agreement claim; the primary CCD analysis does not depend on this sample.

C.4 Scope of the Post-Training Intervention

The paired design is controlled at evaluation time, but the underlying SFT interventions do not follow one standardized recipe. Adapter scaling, dropout, training budget, and data construction vary,

586 and some model–task blocks share an adapter. The 55 evaluation blocks are therefore neither 55
587 independent training runs nor repeated estimates of a common treatment.

588 All fields required to reconstruct training dose—training-set size, maximum steps, train batch size,
589 and gradient accumulation are jointly available for only five of the 55 model–task blocks. Missing
590 settings are left empty rather than imputed, and recovered configurations are reported in Table 8. The
591 study therefore estimates paired changes across the observed interventions; it does not identify a
592 dose–response relation or the causal effect of a standardized SFT procedure.

593 Training targets also contain the same final-answer convention required by the evaluator. Compliance
594 changes therefore describe behavior under an explicitly supervised, evaluation-matched contract;
595 they do not measure transfer to unseen output schemas. CCD asks whether increased adherence to
596 this exposed interface is accompanied by strict success, but its conclusions should not be extended
597 beyond the evaluated contracts.

598 C.5 Standardized Functional Evaluation on MBPP

599 Archived MBPP correctness is not a uniform measure of program behavior: earlier harnesses compare
600 normalized source strings, whereas later versions execute extracted programs. To obtain a common
601 functional criterion, we re-execute every selected candidate for which the task-to-test mapping can
602 be verified. Thirteen checkpoints meet this requirement, producing 117 cells and 11,700 matched
603 program pairs. Gemma 3 270M is excluded because its retained records cannot be linked uniquely to
604 the standardized mapping.

605 Each extracted program is evaluated against the original MBPP assertions, rather than an EvalPlus-
606 augmented test suite [19]. Every candidate remains in the denominator; empty or malformed code,
607 execution errors, timeouts, and failed assertions are all counted as failures. This standardized outcome
608 is C^{func} from Equation (7), and it is evaluated independently of compliance.

609 The distinction from the archived flag is substantive. The full 14-checkpoint pre-SFT MBPP archive
610 contains 12,600 records; 8,100 have no execution status and carry an incorrect archived label. We
611 retain those labels when auditing strict evaluator outcomes, because CCD’s primary question concerns
612 what the evaluator recorded, but use standardized re-execution exclusively for functional claims.
613 Missing historical execution is therefore not treated as evidence that a program fails its tests.

614 For deterministic configurations, functional performance is reported as pass@1. Summaries that
615 also include self-consistency describe the selected program after voting among sampled candidates
616 and are therefore reported as **selected-output functional accuracy**, not conventional pass@1. We
617 additionally test whether any functional change is explained by truncation. A candidate is marked
618 as reaching the generation ceiling when its generated-token count equals the model’s repeated hard
619 upper bound of 256 or 384 tokens; the functional comparison is then repeated after removing every
620 pair with a ceiling hit at either stage.

621 C.6 Analysis Implementation and Provenance

622 Cell-level estimands, equal-cell aggregation, hierarchical bootstrap intervals, paired tests, and robust-
623 ness analyses follow Section 2.5. Their implementation operates entirely on retained paired records
624 and requires no additional model generations. Before analysis, we verify Equation (3) for every
625 response at both stages and reproduce each observation-weighted compliance–success gap from the
626 corresponding transition counts.

627 The provenance audit separates item-level evidence from historical aggregate exports. The archived
628 workbook contains 1,148 rows for 1,008 intended model–task–stage–configuration keys because
629 140 keys occur in more than one export. The audit layer records explicit source precedence, but no
630 paired result is calculated from these competing aggregate rows; all results are recomputed from the
631 item-level files, preventing duplicate exports from entering the analysis.

632 Finally, each of the 55 observed model–task blocks has a manifest recording its source files, pairing
633 key, item count, seed, decoding settings, and SHA-256 file identities. The accompanying materials
634 include the retained generations, 49,140 paired records, execution outcomes for 11,700 MBPP pairs,
635 cell summaries, and analysis scripts. These materials reproduce the evaluation audit, but they do not
636 remove the stated limitation that most historical training runs cannot be reconstructed fully.

637 C.7 Models, run settings, and audit trail

638 Each analytical model label denotes one model used consistently across benchmarks (Table 7). All
639 runs use four-bit weight loading, including checkpoints whose repository names do not specify a
640 quantization format.

Analytical label	Canonical checkpoint / loader ID	Params (B)
Gemma 3 270M	unsloth/gemma-3-270m-it-bnb-4bit	0.27
Gemma 3 1B	unsloth/gemma-3-1b-it-bnb-4bit	1.0
Llama 3 8B	unsloth/llama-3-8b-Instruct-bnb-4bit	8.0
Llama 3.1 8B	unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit	8.0
Llama 3.2 1B	unsloth/Llama-3.2-1B-Instruct-bnb-4bit	1.0
Llama 3.2 3B	unsloth/Llama-3.2-3B-Instruct-bnb-4bit	3.0
Qwen 2.5 0.5B	unsloth/Qwen2.5-0.5B-Instruct-bnb-4bit	0.5
Qwen 2.5 1.5B	unsloth/Qwen2.5-1.5B-Instruct-bnb-4bit	1.5
Qwen 2.5 7B	unsloth/Qwen2.5-7B-Instruct-bnb-4bit	7.0
Qwen 3 0.6B	unsloth/Qwen3-0.6B-unsloth-bnb-4bit	0.6
Qwen 3 1.7B	unsloth/Qwen3-1.7B-unsloth-bnb-4bit	1.7
Qwen 3.5 0.8B	unsloth/Qwen3.5-0.8B	0.8
Qwen 3.5 2B	unsloth/Qwen3.5-2B	2.0
Qwen 3.5 4B	unsloth/Qwen3.5-4B	4.0

Table 7: **One canonical checkpoint per analytical model label.** Dataset-specific strings retained in historical output tables are provenance fields, not distinct checkpoints.

641 C.7.1 Audit trail and recovered run settings

642 The design manifest links each paired model $\times$ dataset evaluation to its source files, row counts,
643 pairing key, seed, and decoding settings; a separate audit layer retains SHA-256 file identities and
644 reconciles historical aggregate exports. For historical auditing only, the aggregate workbook contains
645 1,148 rows for 1,008 intended model $\times$ dataset $\times$ stage $\times$ configuration keys because 140 keys have
646 more than one retained export; the audit layer records explicit source precedence, and all paired
647 results in the paper are computed from item-level files rather than from competing aggregate exports.

648 The completed SFT recipes are heterogeneous. The manifest records each run’s settings where they
649 were recovered and leaves fields explicitly empty rather than imputed otherwise. All four fields needed
650 to reconstruct training dose (training-set size, maximum steps, train batch, gradient accumulation)
651 are available for five of the 55 model–task blocks, including one MBPP block (Table 8); these rows
652 also use LoRA rank and alpha 16/16, learning rate 10^{-4} , train batch 1, and gradient accumulation 16.
653 Effective epochs are $E_{\text{eff}} = \text{steps} \times B_{\text{train}} A_{\text{grad}} / N_{\text{train}}$.

Model	Dataset	Train N	Steps	Max seq	Max new	Eff. epochs	ΔC_{rel}	D_{rel}
Llama 3 8B	StrategyQA	400	220	1024	40	8.8	+1.00	−0.44
Qwen 3.5 0.8B	LogiQA	1500	300	2048	96	3.2	+19.00	+33.44
Qwen 3.5 0.8B	MBPP	300	300	2048	384	16.0	+0.67	+11.89
Qwen 3.5 4B	GSM8K	1500	300	1024	160	3.2	+16.00	+16.22
Qwen 3.5 4B	StrategyQA	500	300	1024	40	9.6	+11.22	−9.78

Table 8: **The five model–dataset recipes with complete dose metadata.** Deltas average the nine inference configurations of each recipe. Five rows, one of them MBPP, do not support a study-wide dose–response analysis: the descriptive Spearman correlation between D_{rel} and effective epochs is $\rho = -.67$ ($p = .22$) and between D_{rel} and maximum steps $\rho = .35$ ($p = .56$), with task and model confounded with dose. We report these values only to show why no dose claim is made. Green shading (lower $\rightarrow$ higher) highlights larger values of D_{rel} only.

D Instrument specification

This appendix states the scoring rules at the level needed to reproduce the (C, F) state assignments. The goal is not to prescribe a universal parser but to make explicit which instrument produced the reported numbers.

GSM8K. The contract requires an explicit `Final Answer:` boundary followed by a number. After comma removal the numeric pattern is `-?\d+(?:\.\d+)?`. With the boundary present, the parser takes the last numeric match inside the boundary region. Without it, the relaxed extractor takes the last numeric match in the full completion and sets $F = 0$. The numeric parser normalizes answer strings by removing commas and a trailing period; they do not generally canonicalize decimal representations such as 2 and 2.0. The analysis retains the recorded labels.

StrategyQA. The contract requires `Final Answer: Yes` or `Final Answer: No`. The strict parser searches the final-answer region for a standalone `yes` or `no`. Without the boundary, the relaxed extractor searches the full completion and returns a recovered label with $F = 0$. The Boolean parser checks `yes` before `no` and defaults to `No` with $F = 0$ when neither is found. Because this can credit an output without an explicit answer, these labels are used only in the parser-derived analysis.

LogiQA. The contract requires one option letter A–D after the final-answer marker. The strict parser searches the answer region for `\b ([ABCD]) \b`; the relaxed path searches the full completion for explicit option letters and uses the recovered option with $F = 0$. Correctness compares normalized letters.

MBPP. The contract requires the final-answer boundary before the returned Python candidate. Without the marker, the extractor retains the final fenced Python block when present, otherwise the completion, and sets $F = 0$. Code-fence markup is stripped and the candidate is trimmed to a probable Python start. The archived correctness label comes from the retained evaluator: earlier implementations use normalized source-string equality, while execution-enabled versions use test outcomes. Execution status is absent for 8,100 of the 12,600 pre-SFT records, which carry an incorrect archived label; the standardized re-execution of Section C.5 runs every extracted candidate of the 13 mapped checkpoints against the original assertions and credits 1,430 pre-SFT and 1,053 post-SFT candidates that the archived flag does not. Non-executable outputs remain failures under both. The archived flag and the re-execution metric are kept separate throughout.

Self-consistency. Each sampled candidate is parsed separately. Earlier evaluators vote over normalized extracted answers (normalized source text for code); if the largest vote counts tie, their helper returns the first sampled prediction. Their format flag is the logical OR over all sampled format flags, including samples that do not support that prediction. Later evaluators choose a most frequent prediction with first-occurrence tie breaking and require a compliant sample among its supporters. Execution-enabled programming evaluators vote using abstract-syntax-tree normalization where parsing succeeds and normalized source otherwise; they prefer a compliant program within the winning group. The archived analysis preserves these distinctions. Thus a self-consistency record describes a harness-level decision and need not represent one raw completion under every version. Greedy-only summaries provide a sensitivity that avoids sample-aggregation differences. Programming summaries that include voted outputs are reported as selected-output functional accuracy rather than conventional pass@1.

E Additional primary and parser-derived summaries

E.1 Cell-level significance counts

Outcome	Δ (pp)	FDR-significant +	FDR-significant –	Total
Compliance F	+34.78	274	16	290
Strict success S	+12.12	176	7	183
Parser-derived C_{rel}	+3.57	54	13	67

Table 9: Equal-cell changes and the number of the 495 cells whose paired change is significant under an exact McNemar test with Benjamini–Hochberg control at $q < .05$, split by sign. Green shading (lower $\rightarrow$ higher) highlights larger changes in the Δ column only.

E.2 Transition taxonomy

Table 10 gives the exhaustive pooled taxonomy behind Figure 4, and Table 11 gives raw counts by benchmark. LogiQA has the largest joint-gain count, StrategyQA the most trade-offs, and MBPP’s parser-level view is dominated by unchanged items and pure compliance gains, which motivates the separate execution analysis. GSM8K is the one benchmark whose pure correctness-loss share (7.76%) exceeds its pure correctness-gain share (7.05%), even though its strict success still rises because compliance gains are far more common.

Category	Transitions	Count	Share
Pure compliance gain	$E_1 \rightarrow E_2, E_3 \rightarrow E_4$	12,228	24.88%
Pure correctness gain	$E_1 \rightarrow E_3, E_2 \rightarrow E_4$	3,128	6.37%
Joint gain	$E_1 \rightarrow E_4$	3,494	7.11%
Pure compliance loss	$E_2 \rightarrow E_1, E_4 \rightarrow E_3$	446	0.91%
Pure correctness loss	$E_3 \rightarrow E_1, E_4 \rightarrow E_2$	2,894	5.89%
Joint loss	$E_4 \rightarrow E_1$	111	0.23%
Trade-off	$E_2 \leftrightarrow E_3$	1,970	4.01%
Unchanged	diagonal	24,869	50.61%

Table 10: **Exhaustive pooled taxonomy over 49,140 matched outputs under the relaxed extractor.** Categories are mutually exclusive and exhaustive. Green shading (lower $\rightarrow$ higher) highlights larger transition shares; color indicates frequency, not improvement.

Benchmark	F gain	C gain	Joint gain	Trade-off	F loss	C loss	Joint loss	Unchanged
GSM8K	2,514	875	569	217	282	964	81	6,918
LogiQA	3,221	965	1,579	575	42	820	12	4,486
MBPP	3,654	169	191	210	63	216	3	8,094
StrategyQA	2,839	1,119	1,155	968	59	894	15	5,371

Table 11: **Raw transition counts by benchmark.** MBPP uses the parser-level correctness label in this cross-task view. Green shading (lower $\rightarrow$ higher) highlights larger counts of pure compliance gains in the F gain column.

E.3 Configuration, headroom, model, and benchmark sensitivities

Configuration	ΔS	ΔF	G	ΔC_{rel}	D_{rel}
Zero	+9.80	+29.18	+19.39	+2.36	+26.82
CoT	+14.40	+35.74	+21.35	+4.65	+31.10
FS0	+17.75	+47.00	+29.25	+5.52	+41.48
FS1	+14.40	+45.61	+31.21	+3.80	+41.81
FS3	+11.35	+39.13	+27.78	+3.16	+35.97
FS5	+6.01	+20.98	+14.97	+1.05	+19.93
SC1	+11.65	+34.48	+22.84	+3.20	+31.28
SC3	+12.14	+31.49	+19.35	+4.17	+27.31
SC5	+11.60	+29.45	+17.85	+4.18	+25.27

Table 12: **Strict and parser-derived contrasts by inference configuration (pp; 55 cells each).** FS0 is the empty-demonstration few-shot scaffold, not direct zero-shot prompting. The compliance gain is largest for FS0 and FS1 and smallest for FS5, whose demonstrations already induce the contract before SFT; both gaps are positive in every configuration. Green shading (lower $\rightarrow$ higher) highlights larger values of the primary gap G only.

Leaving one model out at a time (Table 3) gives $\Delta S \in [11.55, 12.57]$, $\Delta F \in [30.39, 36.84]$, $G \in [18.84, 24.27]$, $\Delta C_{\text{rel}} \in [2.86, 3.96]$, and $D_{\text{rel}} \in [27.11, 33.33]$ pp; the smallest gaps arise when Gemma 3 270M is omitted and the largest when Llama 3 8B is omitted. Weighting changes the picture even less: equal-cell, observation-weighted, and equal-model summaries give $D_{\text{rel}} = 31.22$, 31.10, and 31.81 pp.

Pre-SFT F quartile	n	Mean pre-SFT F	ΔF	D_{rel}
Q1 (lowest)	124	0.56	+71.61	+66.63
Q2	126	32.08	+57.65	+51.43
Q3	122	86.20	+9.67	+8.10
Q4 (highest)	123	99.52	−0.85	−2.26

Table 13: **Headroom dependence (pp)**. Compliance gains, and hence the gap, are concentrated in cells with low pre-SFT compliance and vanish where compliance is already saturated. Green shading (lower $\rightarrow$ higher) highlights larger values of D_{rel} only.

Model	ΔS	ΔF	G	ΔC_{rel}	D_{rel}
Gemma 3 270M	+19.42	+90.83	+71.42	+7.28	+83.56
Gemma 3 1B	+7.78	+24.92	+17.14	−1.47	+26.39
Llama 3 8B	+6.39	+8.58	+2.19	+4.31	+4.28
Llama 3.1 8B	+16.64	+26.44	+9.81	+12.61	+13.83
Llama 3.2 1B	+14.78	+50.14	+35.36	−1.14	+51.28
Llama 3.2 3B	+8.72	+26.17	+17.44	+0.72	+25.44
Qwen 2.5 0.5B	+17.47	+52.08	+34.61	+0.81	+51.28
Qwen 2.5 1.5B	+15.00	+34.61	+19.61	+2.81	+31.81
Qwen 2.5 7B	+18.55	+67.31	+48.76	+3.04	+64.27
Qwen 3 0.6B	+6.36	+14.75	+8.39	+0.81	+13.94
Qwen 3 1.7B	+9.17	+31.47	+22.31	−1.47	+32.94
Qwen 3.5 0.8B	+6.75	+15.92	+9.17	+4.81	+11.11
Qwen 3.5 2B	+11.81	+32.94	+21.14	+6.69	+26.25
Qwen 3.5 4B	+12.47	+18.94	+6.47	+10.00	+8.94

Table 14: **Per-model equal-cell strict and parser-derived contrasts (pp)**, averaged over the observed benchmark–configuration cells of each checkpoint. $G > 0$ and $D_{\text{rel}} > 0$ for every checkpoint. Green shading (lower $\rightarrow$ higher) highlights larger values of the primary gap G only.

E.4 Within-benchmark coupling of compliance and correctness

Figure 6 visualizes cell-level coupling, Table 16 reports the benchmark-level statistics, and Table 17 stratifies correctness changes by the size of the compliance change.

E.5 Extractor sensitivity

Table 18 repeats the pooled contrast under three correctness rules, and Table 19 splits the strict and current rules by benchmark. The largest attenuation occurs on StrategyQA, where a Yes/No answer can often be recovered from explanatory prose even when the required line is missing; the gap nevertheless remains positive under every rule on every benchmark. Section 4.2 derives the general bound $D_{\text{rel}} \geq 11.72$ pp that holds for any extractor.

Table 20 checks whether the parser-derived result is driven by limited pre-SFT correctness headroom.

E.6 Alternative uncertainty constructions

Treating the realized cells as fixed and resampling item pairs gives narrow intervals ([11.75, 12.51] pp for ΔS , [3.15, 4.00] for ΔC_{rel} , [34.51, 35.06] for ΔF); we do not interpret these as population-level uncertainty. A cell-level mixed model with a random intercept per model gives $\Delta S = +12.33$ pp (95% CI [9.75, 14.91]), $\Delta C_{\text{rel}} = +3.70$ ([1.41, 5.99]), and $\Delta F = +35.61$ ([23.52, 47.71]). Table 21 compares the hierarchical bootstrap with a model-cluster bootstrap for the parser-derived contrast on three subsets.

E.7 Exploratory explanatory regression

Table 22 regresses the cell-level D_{rel} on benchmark and configuration, with model family and $\log_{10}$ parameter count as controls, using standard errors clustered by model. With 14 clusters the

Omitted benchmark	Cells	ΔS	ΔF	G	ΔC_{rel}	D_{rel}
GSM8K	369	+14.84	+38.68	+23.83	+4.20	+34.47
LogiQA	378	+10.32	+31.51	+21.18	+1.63	+29.88
MBPP	369	+15.32	+35.91	+20.59	+4.91	+31.00
StrategyQA	369	+8.04	+33.12	+25.08	+3.57	+29.56

Table 15: **Leave-one-benchmark-out sensitivity (pp)**. Strict columns follow from the per-benchmark equal-cell means in Table 1. Both gaps remain positive after removing any task family. Green shading (lower $\rightarrow$ higher) highlights larger values of the primary gap G only.

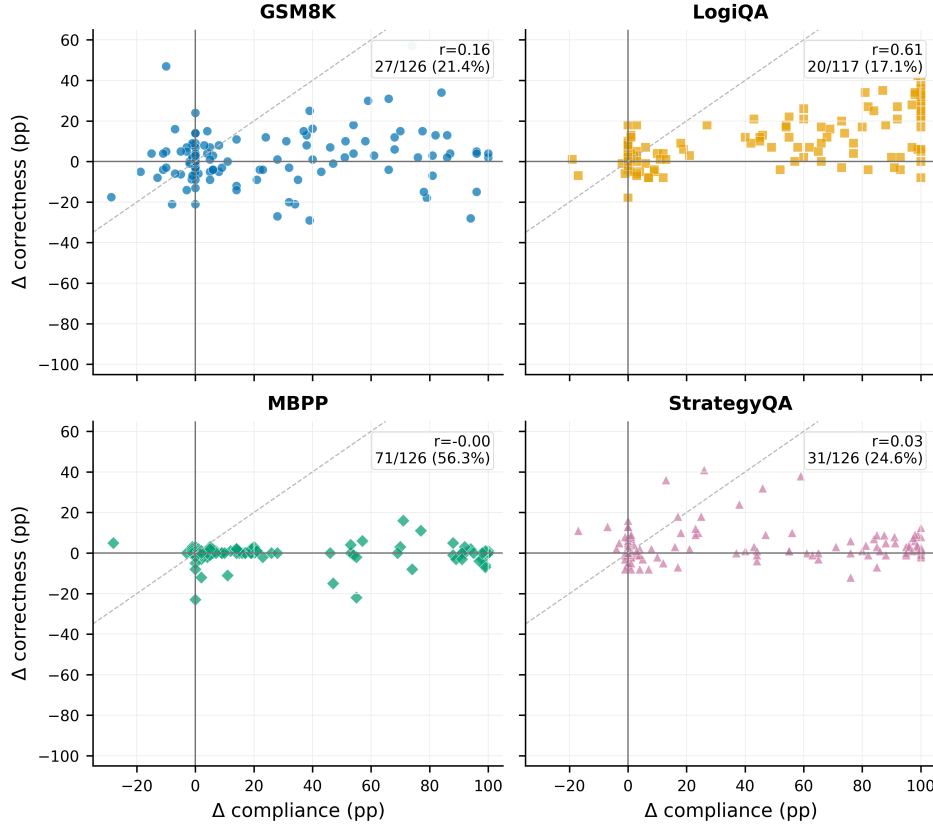


Figure 6: **Cell-level changes in compliance (ΔF) and parser-derived correctness (ΔC_{rel}) for the 495 cells, by benchmark.** Annotations give the Pearson correlation and the number of cells with $\Delta F > 0$ and $\Delta C_{\text{rel}} \leq 0$; the dashed diagonal marks equal changes.

coefficients are interpreted as broad associations only. The useful descriptive pattern is task and configuration heterogeneity; family and size controls have very wide intervals and are omitted from the table.

F MBPP execution details

Generation-ceiling audit. The ceiling proxy is computed on deterministic configurations, where per-candidate generated-token counts are available; self-consistency aggregates tokens across samples and is excluded from this check by design. Across the 13 checkpoints, 1,301 of 7,800 pre-SFT candidates (16.68%) and 89 of 7,800 post-SFT candidates (1.14%) hit their model’s empirical ceiling (256 or 384 tokens). Excluding pairs with a hit at either stage leaves 6,436 pairs; the model-macro change moves from -4.82 pp (CI $[-9.65, -1.05]$) to -5.47 pp (CI $[-10.57, -1.30]$). This does not rule out every length-related mechanism, but it rules out the specific explanation that the decline is driven by more frequent truncation after SFT.

Benchmark	Cells	Pearson r	Spearman ρ	$\Delta F > 0, \Delta C_{\text{rel}} \leq 0$	Mean D_{rel} (pp)
GSM8K	126	.16	.18	27/126 (21.4%)	21.68
LogiQA	117	.61	.55	20/117 (17.1%)	35.56
MBPP	126	-.00	-.11	71/126 (56.3%)	31.86
StrategyQA	126	.03	.12	31/126 (24.6%)	36.09
Pooled	495	.28	.24	149/495 (30.1%)	31.22

Table 16: **Within-benchmark coupling between ΔF and ΔC_{rel} .** On the 117-cell functional MBPP subset the corresponding values are $r = .23$, $\rho = .16$, 64/117 cells (54.7%) with $\Delta F > 0$ and $\Delta C_{\text{func}} \leq 0$, and mean $D_{\text{func}} = 30.87$ pp. Partial correlations that control for pre-SFT levels are .23, and .17 with benchmark fixed effects. Green shading (lower $\rightarrow$ higher) highlights larger percentages of cells with $\Delta F > 0$ and $\Delta C_{\text{rel}} \leq 0$; other columns are unshaded.

ΔF quartile	n	Mean ΔF	Mean ΔC_{rel}
Q1 (lowest)	124	-2.34	+0.88
Q2	124	+4.31	+0.40
Q3	123	+44.00	+5.93
Q4 (highest)	124	+93.23	+7.07

Table 17: **Parser-derived correctness change conditional on the size of the compliance change (pp).** Even the cells with the largest compliance gains show correctness gains an order of magnitude smaller. Green shading (lower $\rightarrow$ higher) highlights larger mean correctness changes ΔC_{rel} only.

G Exact transition accounting

For a matched item with pre- and post-SFT states (C_0, F_0) and (C_1, F_1) under an instrument C , let $\Delta C = C_1 - C_0$ and $\Delta F = F_1 - F_0$. The joint outcome $J_C = CF$ satisfies $\Delta J_C = F_0 \Delta C + C_0 \Delta F + \Delta C \Delta F$ (Eq. 8). J_C equals archived strict success S exactly when $C = C_{\text{rel}}$ (Eq. 3); under C_{func} it is the joint outcome of compliance and standardized re-execution, which differs from the archived S on MBPP (Section 2.4). For states with $F = 0$ the identity inherits the extractor’s dependence. Table 27 lists the coordinate changes of the representative transitions. The same compliance change affects the joint score differently depending on the initial correctness state: a boundary repair on an already-correct item ($E_3 \rightarrow E_4$) raises the joint score, whereas the same repair on a wrong item ($E_1 \rightarrow E_2$) does not. The transition matrix therefore carries strictly more information than D_C alone: D_C describes the relative movement of two marginal rates, while the matrix identifies which joint states generated it.

H Machine-readable reporting schema

CCD requires only a small extension to ordinary benchmark logging. Table 28 lists the fields needed to reproduce the decomposition and to audit extractor decisions; retaining the raw completion allows the contract checker and any relaxed extractor to be re-run without regenerating model outputs. A minimal implementation follows six steps: generate and store the raw completion; apply the contract checker; apply the strict task criterion; apply each relaxed instrument independently of the contract result; store S and any instrument-specific joint outcome $J_C = FC$ separately; and pair records by stable item ID before computing transitions. Under self-consistency, the same logic is applied to each sampled candidate before the vote winner and its format flag are selected. The ordering matters: conditioning relaxed extraction on contract validity would make the two axes mechanically dependent.

I Qualitative paired examples

The six examples below were selected purposively from the item-level records to illustrate distinct transition mechanisms; they are not a random sample and carry no prevalence information. Non-MBPP state labels are exact under the documented extractors but are not human-adjudicated; MBPP examples use benchmark execution status. Figures 7, 8, and 9 show compliance-only, correctness-only, and execution-based transitions, respectively.

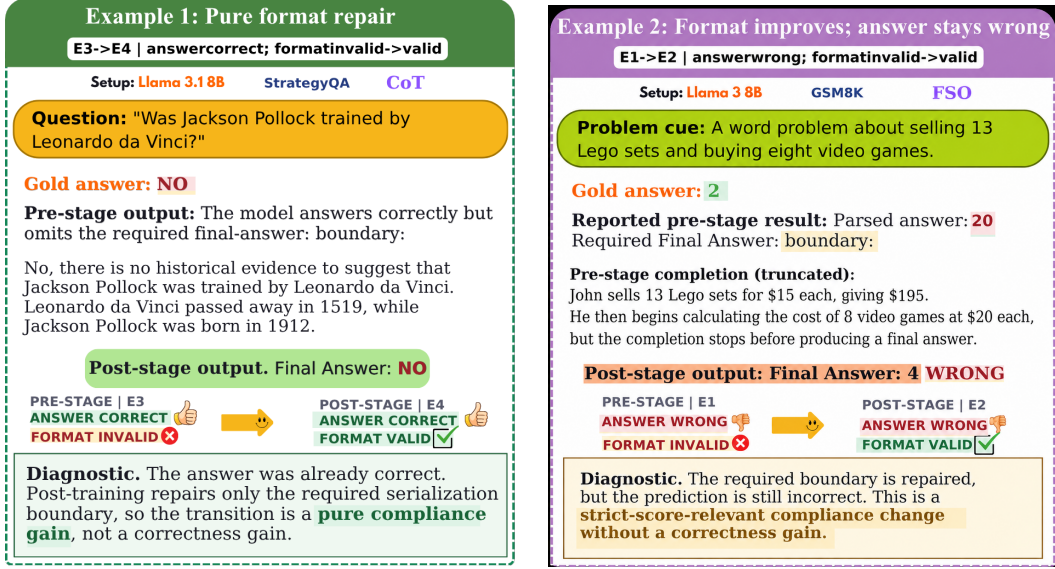


Figure 7: **Compliance repairs.** Left: an already-correct StrategyQA response becomes contract-valid ($E_3 \rightarrow E_4$), which raises the strict score without any change in correctness. Right: a GSM8K response remains incorrect while the required final-answer boundary is repaired ($E_1 \rightarrow E_2$), the single most common transition in the study.

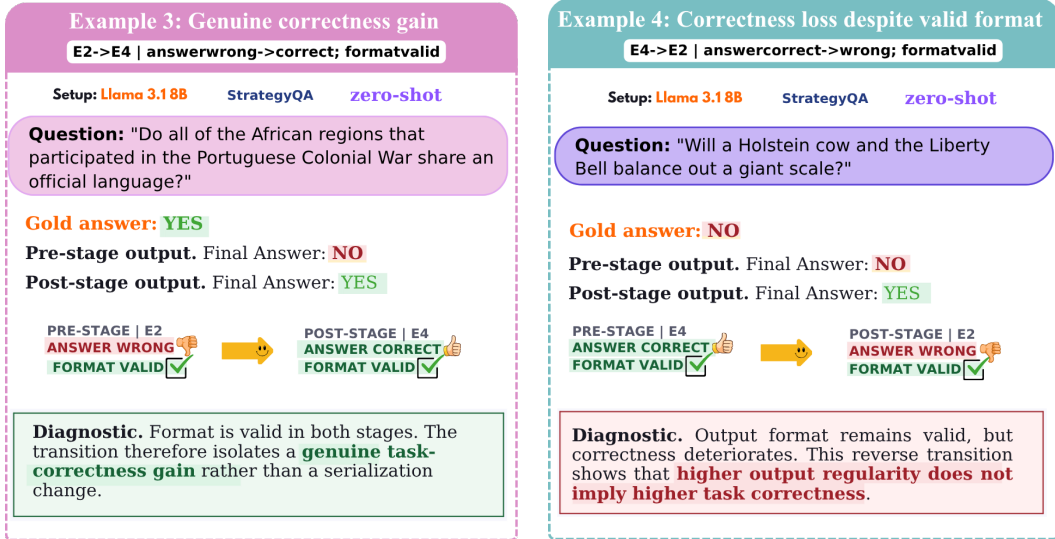


Figure 8: **Correctness changes at constant compliance.** Left: incorrect to correct while remaining format-valid ($E_2 \rightarrow E_4$). Right: correct to incorrect while remaining format-valid ($E_4 \rightarrow E_2$). Improved output regularity carries no information about which of these occurred.

Correctness rule applied to invalid outputs	ΔC	ΔF	D_C
Strict (never credited)	+12.12	+34.78	+22.66
Current relaxed extractor	+3.57	+34.78	+31.22
Maximally lenient bound (always credited)	-22.66	+34.78	+57.45

Table 18: **Pooled extractor sensitivity (pp)**. The maximally lenient row is an algebraic bound, not an operational parser. Green shading (lower $\rightarrow$ higher) highlights larger values of D_C only.

Benchmark	ΔC		ΔF	D_C	
	Relaxed	Strict		Relaxed	Strict
GSM8K	+1.70	+4.14	+23.38	+21.68	+19.24
LogiQA	+9.82	+17.92	+45.38	+35.56	+27.45
MBPP [†]	-0.37	+2.76	+31.48	+31.86	+28.72
StrategyQA	+3.56	+24.07	+39.65	+36.09	+15.59

Table 19: **Extractor sensitivity by benchmark (pp)**. ΔF is unchanged because only the correctness rule varies. [†]MBPP uses the parser-level label in this cross-task diagnostic; execution results are in Section 4.3. Only the two D_C columns are shaded; green (lower $\rightarrow$ higher) encodes larger gaps on a shared scale.

Pre-SFT C_{rel} quartile	n	Mean pre-SFT C_{rel}	ΔC_{rel}	ΔF
Q1 (lowest)	125	0.59	+2.30	+33.39
Q2	124	18.42	+8.28	+46.04
Q3	137	42.28	+3.05	+34.62
Q4 (highest)	109	60.03	+0.30	+23.78

Table 20: **Changes after stratifying cells by pre-SFT parser-derived correctness (pp)**. Correctness has ample headroom in every quartile, yet its change remains small relative to compliance. Green shading (lower $\rightarrow$ higher) highlights larger correctness changes ΔC_{rel} only.

Subset	D_{rel}	Hierarchical 95% CI	Model-cluster 95% CI
Deterministic	32.85	[18.42, 49.20]	[22.19, 45.04]
Self-consistency	27.95	[10.62, 48.00]	[16.75, 41.82]
Non-MBPP	31.00	[17.99, 45.82]	[20.74, 42.64]

Table 21: **Parser-derived contrast under two uncertainty constructions (pp)**. Green shading (lower $\rightarrow$ higher) highlights larger point estimates of D_{rel} ; intervals are unshaded.

Predictor	Association with D_{rel} (pp)	Cluster-robust 95% CI
Intercept	+40.20	[1.72, 78.67]
LogiQA vs. GSM8K	+12.75	[0.25, 25.25]
MBPP vs. GSM8K	+10.17	[-8.26, 28.61]
StrategyQA vs. GSM8K	+14.41	[2.19, 26.63]
FS0 vs. CoT	+10.38	[1.35, 19.41]
FS1 vs. CoT	+10.72	[2.17, 19.27]
FS3 vs. CoT	+4.87	[-7.12, 16.86]
FS5 vs. CoT	-11.17	[-25.75, 3.41]
SC1 vs. CoT	+0.18	[-2.92, 3.28]
SC3 vs. CoT	-3.78	[-8.34, 0.78]
SC5 vs. CoT	-5.82	[-10.51, -1.13]
Zero vs. CoT	-4.27	[-16.60, 8.06]

Table 22: **Exploratory OLS of cell-level D_{rel} on benchmark and configuration (reference levels GSM8K, CoT, Gemma), 495 cells, 14 model clusters, $R^2 = .158$** . Green shading (lower $\rightarrow$ higher) highlights larger signed coefficients among the contrasts; color does not indicate significance.

Configuration	Pre	Post	ΔC_{func}	ΔF	D_{func}
Zero	21.23	18.00	-3.23	+32.00	+35.23
CoT	21.23	18.69	-2.54	+32.62	+35.15
FS0	19.54	15.23	-4.31	+26.00	+30.31
FS1	21.00	14.31	-6.69	+28.92	+35.62
FS3	21.46	14.77	-6.69	+26.38	+33.08
FS5	13.23	7.77	-5.46	+11.92	+17.38
SC1	18.31	15.62	-2.69	+31.77	+34.46
SC3	17.23	15.85	-1.38	+28.92	+30.31
SC5	17.69	17.77	+0.08	+26.38	+26.31

Table 23: **Functional MBPP results by configuration, averaged over the 13 executable checkpoints (%; changes in pp).** $D_{\text{func}} = \Delta F - \Delta C_{\text{func}}$ is positive in every configuration; SC5 is the only configuration whose functional change is non-negative. Green shading (lower $\rightarrow$ higher) highlights larger signed functional changes ΔC_{func} only; color does not indicate significance.

Model	Pre	Post	ΔC_{func}
Gemma 3 1B	2.67	2.00	-0.67
Llama 3 8B	55.44	32.11	-23.33
Llama 3.1 8B	1.89	1.00	-0.89
Llama 3.2 1B	0.00	0.22	+0.22
Llama 3.2 3B	0.67	0.89	+0.22
Qwen 2.5 0.5B	16.00	15.44	-0.56
Qwen 2.5 1.5B	35.11	35.00	-0.11
Qwen 2.5 7B	4.56	5.67	+1.11
Qwen 3 0.6B	2.33	3.67	+1.33
Qwen 3 1.7B	42.78	36.22	-6.56
Qwen 3.5 0.8B	1.00	1.67	+0.67
Qwen 3.5 2B	30.89	23.11	-7.78
Qwen 3.5 4B	53.56	42.33	-11.22

Table 24: **Selected-output functional accuracy (%) averaged over all nine configurations for each executable checkpoint, and its change (pp).** Green shading (lower $\rightarrow$ higher) highlights larger signed functional changes ΔC_{func} only.

Model	Pre	Post	Δ	Paired 95% CI	Δ non-hit	n non-hit
Llama 3 8B	54.83	27.17	-27.67	[-31.67, -23.50]	-28.09	591
Qwen 3.5 2B	35.33	22.50	-12.83	[-17.00, -8.67]	-17.26	336
Qwen 3.5 4B	51.67	40.33	-11.33	[-14.67, -7.83]	-12.24	531
Qwen 3 1.7B	42.50	37.17	-5.33	[-9.50, -1.17]	-4.73	571
Qwen 2.5 1.5B	39.50	34.67	-4.83	[-9.00, -0.50]	-7.14	420
Qwen 2.5 0.5B	17.83	15.00	-2.83	[-6.50, 0.50]	-3.59	474
Llama 3.1 8B	2.50	0.83	-1.67	[-3.17, -0.17]	-1.78	507
Gemma 3 1B	1.83	1.33	-0.50	[-1.83, 0.83]	-0.52	578
Llama 3.2 1B	0.00	0.33	+0.33	[0.00, 0.83]	+0.00	183
Llama 3.2 3B	1.00	1.33	+0.33	[-0.50, 1.33]	+0.36	548
Qwen 3.5 0.8B	1.17	2.00	+0.83	[-0.17, 2.00]	+0.95	526
Qwen 2.5 7B	4.67	5.67	+1.00	[-1.17, 3.33]	+1.04	576
Qwen 3 0.6B	2.17	4.00	+1.83	[0.50, 3.33]	+1.85	595

Table 25: **Numbers behind Figure 5: deterministic pass@1 (%), change (pp), paired-bootstrap 95% interval over 600 pairs per checkpoint, and the change after excluding pairs that reach the generation-ceiling proxy at either stage.** Green shading (lower $\rightarrow$ higher) highlights larger signed changes in the primary Δ column; color does not indicate significance.

Execution outcome class	Pre-SFT	Post-SFT
Passed (detailed trace)	2,192	1,746
Pass/fail retained from saved execution label	1,800	1,800
Test assertion failure	1,992	2,278
Runtime or test exception	3,099	2,590
Syntax error	2,511	3,189
Missing code	69	28
Timeout	20	44
Blocked by safety harness	17	25

Table 26: **Marginal execution outcome classes for the 11,700 candidates per stage.** Every class remains in the denominator; the paired view in Table 4B is more informative than these marginals. Green shading (lower $\rightarrow$ higher) highlights larger post-SFT counts only; color indicates frequency, not improvement.

Category	Move	ΔC	ΔF	ΔJ_C
Pure compliance gain	$E_1 \rightarrow E_2$	0	+1	0
Pure compliance gain	$E_3 \rightarrow E_4$	0	+1	+1
Pure correctness gain	$E_1 \rightarrow E_3$	+1	0	0
Pure correctness gain	$E_2 \rightarrow E_4$	+1	0	+1
Joint gain	$E_1 \rightarrow E_4$	+1	+1	+1
Trade-off	$E_3 \rightarrow E_2$	-1	+1	0
Trade-off	$E_2 \rightarrow E_3$	+1	-1	0
Pure correctness loss	$E_4 \rightarrow E_2$	-1	0	-1
Joint loss	$E_4 \rightarrow E_1$	-1	-1	-1

Table 27: **Coordinate changes of representative transitions under an instrument C .**

Field	Level	Purpose
item_id	item	Stable pre/post matching key
model_label	run	Analytical checkpoint identity
dataset	run	Task family
configuration	run	Zero / CoT / FSK / SK condition
stage	item	Pre- or post-training
completion	item	Raw generated text
extracted_answer	item	Answer returned by the task extractor
instrument_id	run	Versioned contract checker and extractor definition
format_ok	item	Contract-compliance indicator F
correct	item	Correctness label under instrument_id
strict_success	item	Directly observed strict success S
execution_status	item	Behavioral result and error class when executable
generated_tokens	item	Length and generation-ceiling diagnostics

Table 28: **Minimal fields for reproducing CCD from evaluation logs.**

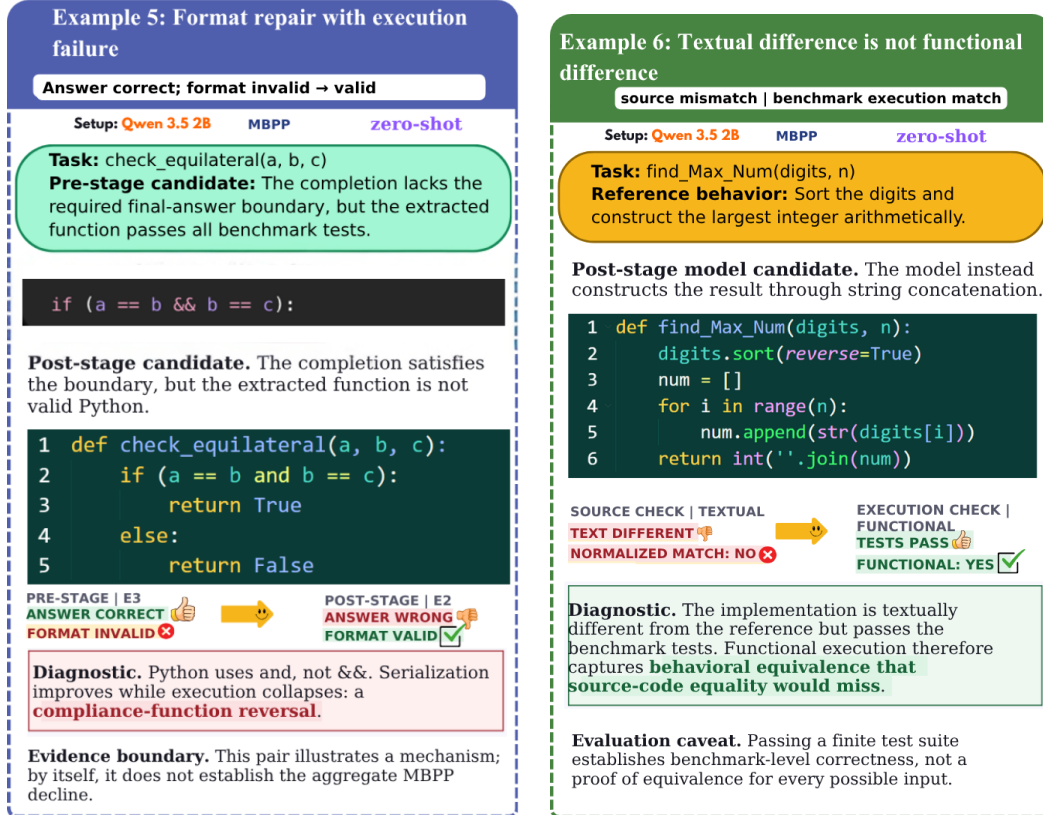


Figure 9: **Execution-based MBPP examples.** Left: compliance improves while a passing pre-SFT program is replaced by a boundary-compliant post-SFT program that is not valid Python (&& in place of and), the mechanism behind the syntax-error channel in Table 4B. Right: a generated implementation differs textually from the reference yet passes the benchmark tests, illustrating why execution rather than source agreement anchors programming claims. Passing a finite test suite establishes benchmark-level correctness, not equivalence on every input.